



東華大學
DONGHUA UNIVERSITY

多维数据结构与运算

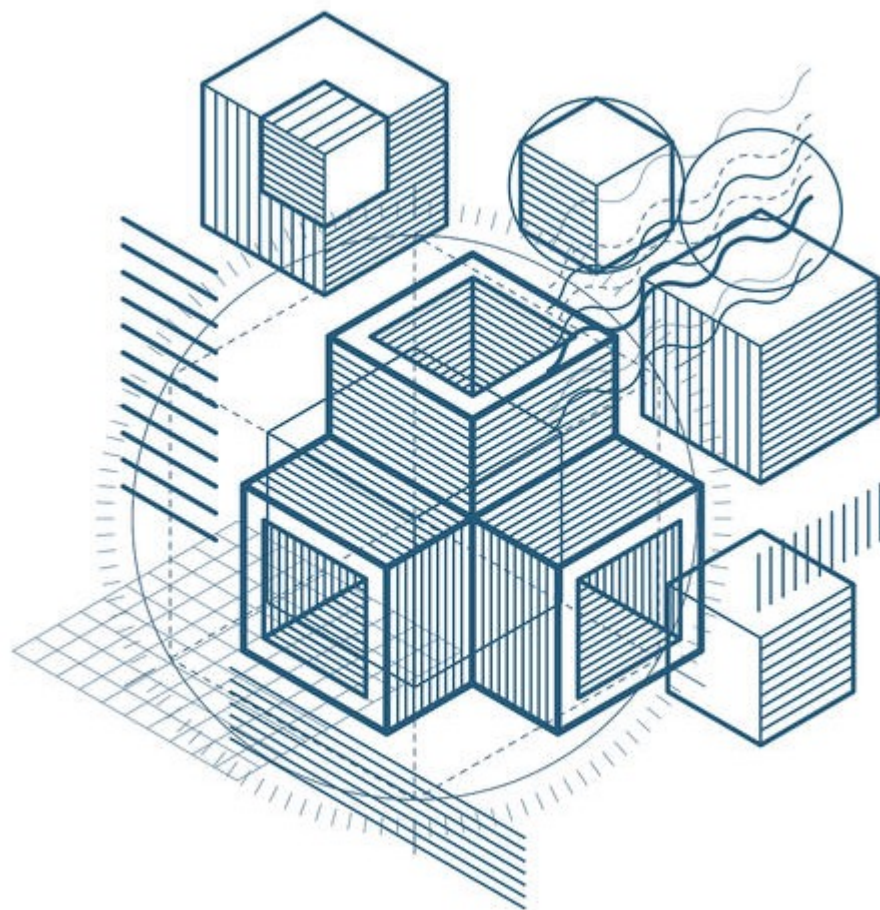
DATA

计算机科学与技术学院

主讲教师：王洪亚、宋晖

2.1 多维数据对象

- 多维数据
- Numpy array
- 一维数据对象
- 二维数据对象



数据和数据维度

- 在数据分析应用中经常需要从多个维度来表示数据
 - 一维数据、二维数据
- 维度（dimension）称为轴（axis）
 - 0维数据（标量），一个数字，12，34.8
 - 1维数据（向量），一个班级同学的姓名，name[5]

'王涛'	'李利茗'	'张广量'	'赵祁姗'	'吴孟'
------	-------	-------	-------	------

姓名

多维数据

- **2维数据(矩阵)**, 数据集最常见的形式

- 人口统计数据集

- 第0轴为样本
 - 第1轴为特征

0轴 样本

姓名	出生年	常住城市	年收入	职业
'李利茗'	1980	'苏州'	30.5	'医生 '
'张广量'	1978	'广州'	25.6	'教师 '
'赵祁姗'	1989	'北京'	15.8	'公务员 '
'吴孟'	1992	'上海 '	18.6	'工程师 '

1轴 样本特征

- **3维数据**, 常用于存储时间或序列数据

- 零售店每天各商品销量数据集

- 第0轴: 门店
 - 第1轴: 商品
 - 第2轴: 时间

89	134	3	78	561
67	178	5	34	452
72	108	1	12	523
73	204	6	46	491

0轴 门店

1轴 商品

2轴 时间

多维数据表示

- 基础Python库不支持多维数组
 - 引入Numpy
- NumPy库提供了多维数组对象ndarray
 - 支持丰富数据表示方式
 - Anaconda环境中已安装

```
>>> import numpy as np
```

- ndarray: NumPy核心
 - N-dimensional array, N维数组
 - 相同数据类型的元素组成的多维数组
 - 数组大小需事先指定, 且不可改变

案例2-1：学生课程考试成绩

- 5位同学参加了学业水平考试，考试科目共7门

姓名	Math	English	Python	Chinese	Art	Database	Physics
王微	70	85	77	90	82	84	89
肖良英	60	64	80	75	80	92	90
方绮雯	90	93	88	87	86	90	91
刘旭阳	80	82	91	88	83	86	80
钱易铭	88	72	78	90	91	73	80

2.1.2 一维数组对象

- 创建一维数组分别保存学生姓名和考试科目，访问数组元素
 - `np.array()`，基于列表创建一维数组

```
>>> names = np.array(['王微', '肖良英', '方绮雯', '刘旭阳', '钱易铭'])
>>> names
array(['王微', '肖良英', '方绮雯', '刘旭阳', '钱易铭', dtype='<U3')
>>> subjects = np.array(['Math', 'English', 'Python', 'Chinese', 'Art', 'Database', 'Physics'])
>>> subjects
array(['Math', 'English', 'Python', 'Chinese', 'Art', 'Database', 'Physics'], dtype='<U8')
```

一维数组访问

```
>>> names
array(['王微', '肖良英', '方绮雯', '刘旭阳', '钱易铭', dtype='<U3')
>>> subjects
array(['Math', 'English', 'Python', 'Chinese', 'Art', 'Database',
      'Physics'], dtype='<U8')
```

- 数组特性

```
>>> names.ndim    #数组维度
1
>>> names.size    #数组元素个数
5
>>> names.dtype    #数组数据类型
dtype('<U3')
```

- 索引序号范围为[0, n-1]或[-n, -1]
 - n 为数组大小
 - 与Python序列的索引使用方式一致

```
>>> names[2]
'方绮雯'
>>> subjects[-3]
'Art'
```


一维数组切片

```
>>> names
array(['王微', '肖良英', '方绮雯', '刘旭阳', '钱易铭', dtype='<U3')
>>> subjects
array(['Math', 'English', 'Python', 'Chinese', 'Art', 'Database',
'Physics'], dtype='<U8')
```

- 抽取数组的一部分元素生成新数组称为切片操作
 - 直接以列表形式给出需要的元素索引

```
>>> subjects[ [0,2,4] ]      #两层括号, [0,2,4]为索引列表
array(['Math', 'Python', 'Art'], dtype='<U8')
```

外层括号表示元素访问
内层括号表示列表

- 索引也可表示为 : start:end:step
 - 生成 start ~ end-1, 步长step的等差数列

```
>>> names[ 1:4 ]   #抽取索引为1、2、3的元素
array(['肖良英', '方绮雯', '刘旭阳'], dtype='<U3')
>>> subjects[ : -1:2 ] #抽取索引为0、2、4的元素
array(['Math', 'Python', 'Art'], dtype='<U8')
```

start省略 : 从0开始
end省略 : 到最后
step省略 : 步长为1

根据条件筛选数组元素

- 使用条件表达式和关系运算符

- 筛选出names数组中值等于“王微”或“钱易铭”的元素

```
>>> names[ (names == '王微') | (names== '钱易铭')]  
array(['王微', '钱易铭'], dtype='<U3')
```

- 条件表达式：生成布尔型的数组

```
>>> mask = (names == '王微') | (names== '钱易铭')  
>>> mask  
array([ True, False, False, False, True ], dtype=bool)  
>>> names[ mask ]  
array(['王微' '钱易铭'], dtype='<U3')
```

names	'王微'	'肖良英'	'方绮雯'	'刘旭阳'	'钱易铭'
mask	True	False	False	False	True
names[mask]	array(['王微', '钱易铭'], dtype='<U3')				

2.1.3 二维数组对象

- 创建二维数组scores，记录 “names” 中同学 “subjects” 的各门课程考试成绩

```
>>> scores= np.array([[70,85,77,90,82,84,89],  
[60,64,80,75,80,92,90], [90,93,88,87,86,90,91],  
[80,82,91,88,83,86,80],[88,72,78,90,91,73,80]])
```

```
>>> scores
```

```
array([[70, 85, 77, 90, 82, 84, 89],  
       [60, 64, 80, 75, 80, 92, 90],  
       [90, 93, 88, 87, 86, 90, 91],  
       [80, 82, 91, 88, 83, 86, 80],  
       [88, 72, 78, 90, 91, 73, 80]])
```

一行数据

- 使用array函数创建二维ndarray数组对象
 - 初始化的列表，其元素也是列表

访问二维数组元素

```
array([[70, 85, 77, 90, 82, 84, 89],
       [60, 64, 80, 75, 80, 92, 90],
       [90, 93, 88, 87, 86, 90, 91],
       [80, 82, 91, 88, 83, 86, 80],
       [88, 72, 78, 90, 91, 73, 80]])
```

- 数组特性

```
>>> scores.ndim    #数组维数
2
>>> scores.size     #数组元素总数，行×列
35
>>> scores.shape    #数组的行数和列数
(5, 7)
>>> scores.dtype    #数组元素的类型
dtype('int32')
```

- 二维数组切片操作的基本格式：**arr[row , column]**
 - row行序号，column列序号，中间用 ‘ , ’ 隔开
 - 行、列切片的表示方式与一维数组相同
 - 用 “:” 代替行或列，表示选中对应的所有行或列

二维数组切片

```
array([[70, 85, 77, 90, 82, 84, 89],
       [60, 64, 80, 75, 80, 92, 90],
       [90, 93, 88, 87, 86, 90, 91],
       [80, 82, 91, 88, 83, 86, 80],
       [88, 72, 78, 90, 91, 73, 80]])
```

- 访问指定行、列的元素，给出行和列两个索引值

```
>>> scores[1,0]
60
```

```
>>> scores[[1,3],[0,1]]
array([60, 82])
```

抽取scores[1,0],scores[3,1]

```
>>> scores[[1,3]]
array([[60, 64, 80, 75, 80, 92, 90],
       [80, 82, 91, 88, 83, 86, 80]])
```

取某行所有列值，可以省略列：

抽取部分列，行不能省略

```
>>> scores[:, [0,1]]
array([[70, 85],
       [60, 64],
       [90, 93],
       [80, 82],
       [88, 72]])
```

```
>>> scores[ [0,3], 1:4 ]
array([85, 77, 90]
       [ 82, 91, 88,])
```

抽取1、3行中0、1列数据，两层切片

```
>>> scores[[1,3]][:,[0,1]]
array([[60, 64],
       [80, 82]])
```

条件筛选

```
array([[70, 85, 77, 90, 82, 84, 89], False
       [60, 64, 80, 75, 80, 92, 90], True
       [90, 93, 88, 87, 86, 90, 91], True
       [80, 82, 91, 88, 83, 86, 80], False
       [88, 72, 78, 90, 91, 73, 80]]) False
True False True False False False
```

- 使用布尔型数组筛选访问**其他数组**的元素

```
>>> scores[(names == '肖良英') | (names == '方绮雯'), :]  
array([[60, 64, 80, 75, 80, 92, 90],  
       [90, 93, 88, 87, 86, 90, 91]])
```

行筛选，names的长度 和 scores的行数一致

- 选择“肖良英”和“方绮雯”的“Math”和“Python”课程成绩，可以使用两层筛选实现

```
>>> scores[(names == '肖良英') | (names == '方绮雯')][:,(subjects ==  
'Math') | (subjects == 'Python')]  
array([[60, 80],  
       [90, 88]])
```

第一层筛选后的所有行，2行

创建多维数组的常用方法

- arange()

```
>>> np.arange(0, 11)
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
>>> np.arange(3, 11, 2)
array([3, 5, 7, 9])
>>> np.arange(0.3, 1.5, 0.3)
array([ 0.3,  0.6,  0.9,  1.2])
```

arange可以生成浮点数序列

- reshape()

- 将一维数组转换为指定的多维数组

```
>>> np.arange(0, 15).reshape(3, 5)
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
>>> np.zeros((3, 4))
array([[ 0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.],
       [ 0.,  0.,  0.,  0.]])
```

- zeros()、ones()

```
>>> np.ones((4, 3))
array([[ 1.,  1.,  1.],
       [ 1.,  1.,  1.],
       [ 1.,  1.,  1.],
       [ 1.,  1.,  1.]])
```

思考与练习

1. 一维数组访问。

- ① 在subjects数组中选择并显示序号1、2、4门课的名称，使用负值序号选择并显示names数组中 '方绮雯'。
- ② 选择并显示names数组从2到最后的数组元素；选择并显示subjects数组2到4的数组元素。
- ③ 使用布尔条件选择并显示subjects数组中的英语和物理科目名称。

2. 二维数组访问。

- ① 选择并显示scores数组的1、4行。
- ② 选择并显示scores数组中行号2、4同学的数学和Python成绩。
- ③ 选择并显示scores数组中所有同学的数学和艺术课程成绩。
- ④ 选择并显示scores数组中“王微”和“刘旭阳”的英语和艺术课程成绩。

3. 生成由整数10~19组成的2×5的二维数组。

课后作业

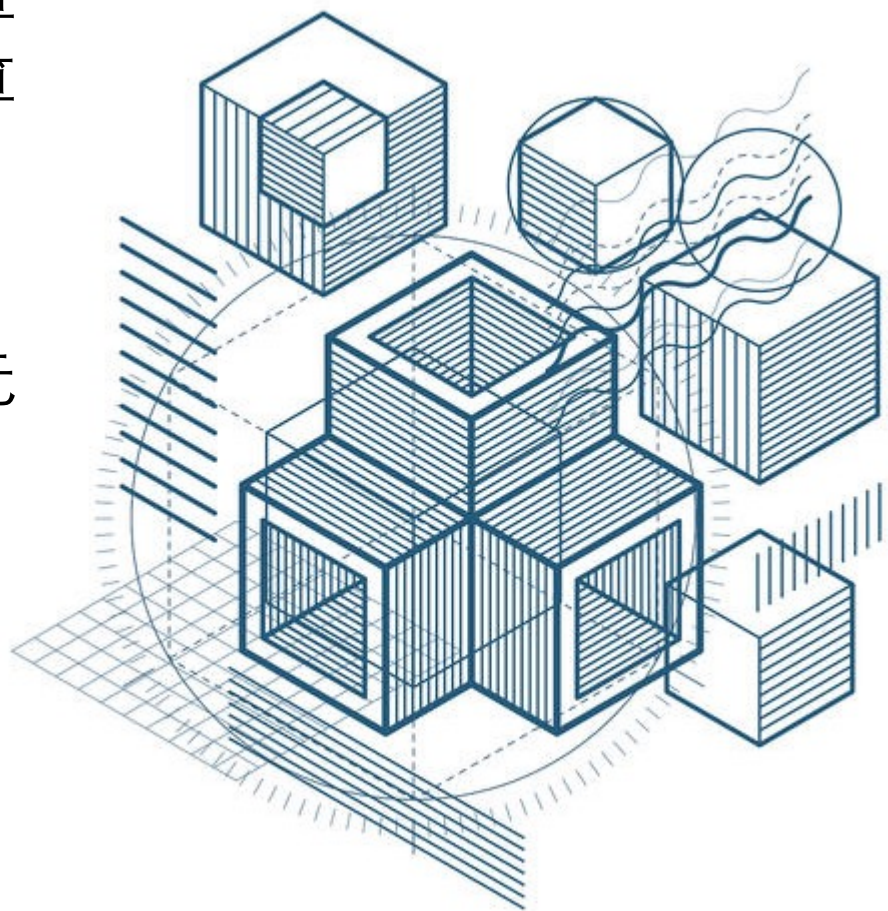
打包提交名为：“学号-姓名-Data-1.zip”

“大润发”、“沃尔玛”、“联华”和“农工商”四个超市都卖苹果、香蕉、桔子和芒果四种水果。使用NumPy的ndarray实现以下功能。

- a) 创建2个一维数组分别存储超市名称和水果名称；
- b) 创建1个 4×4 的二维数组存储不同超市的水果价格，其中价格由4到10范围内的随机数生成；
- c) 选择“大润发”的苹果和“联华”的香蕉，并将价格增加1元；
- d) “农工商”水果大减价，所有水果价格减少2元；
- e) 统计四个超市苹果和芒果的销售均价；
- f) 找出桔子价格最贵的超市名称（不是编号）。

2.2 多维数组运算

- 基本算数运算
 - 标量与数组之间的运算
 - 数组与数组之间的运算
- 函数运算
 - 通用函数：一元、二元
 - 聚合函数
 - 随机生成函数



二维数组与标量运算

- 为所有同学的所有课程成绩增加5分。

```
>>> scores + 5  
array([[75, 90, 82, 95, 87, 89, 94],  
       [65, 69, 85, 80, 85, 97, 95],  
       [95, 98, 93, 92, 91, 95, 96],  
       [85, 87, 96, 93, 88, 91, 85],  
       [93, 77, 83, 95, 96, 78, 85]])
```

```
>>> a = np.ones((5, 7)) * 5
```

```
>>> a
```

```
array([[ 5.,  5.,  5.,  5.,  5.,  5.,  5.],  
       [ 5.,  5.,  5.,  5.,  5.,  5.,  5.],  
       [ 5.,  5.,  5.,  5.,  5.,  5.,  5.],  
       [ 5.,  5.,  5.,  5.,  5.,  5.,  5.],  
       [ 5.,  5.,  5.,  5.,  5.,  5.,  5.]])
```

等价于

```
>>> scores + a
```

```
array([[ 75.,  90.,  82.,  95.,  87.,  89.,  94.],  
       [ 65.,  69.,  85.,  80.,  85.,  97.,  95.],  
       [ 95.,  98.,  93.,  92.,  91.,  95.,  96.],  
       [ 85.,  87.,  96.,  93.,  88.,  91.,  85.],  
       [ 93.,  77.,  83.,  95.,  96.,  78.,  85.]])
```

二维数组与一维数组运算

- 每个科目基础分不同，为各科目增加相应的基础分。

```
>>> bonus = np.array([3,4,5,3,6,7,2])
>>> scores + bonus
array([[73, 89, 82, 93, 88, 91, 91],
       [63, 68, 85, 78, 86, 99, 92],
       [93, 97, 93, 90, 92, 97, 93],
       [83, 86, 96, 91, 89, 93, 82],
       [91, 76, 83, 93, 97, 80, 82]])
```

- 修改选定数据

```
>>> scores[names == '肖良英', subjects == 'English']
array([64])
>>> scores[names == '肖良英', subjects == 'English'] + 5
array([69])
```

多维数组运算-通用函数

- 一元，1个输入数组，返回1个数组

函数	描述
abs、fabs	计算整数、浮点数或复数的绝对值
sqrt	计算各元素的平方根
square	计算各元素的平方
exp	计算各元素的指数
log、log10	自然对数、底数为10的log
sign	计算各元素的正负号
ceil	计算各元素的ceiling值，即大于等于该值的最小整数
floor	计算各元素的floor值，即小于等于该值的最大整数
cos、cosh、sin sinh、tan、tanh	普通和双曲型三角函数

通用函数-二元

- 2个输入数组，返回1个数组

函数	描述
add	将数据中对应的元素相加
subtract	从第一个数组中减去第二个数组中的元素
multiply	数组元素相乘
divide	数组对应元素相除
power	对第一个数组中的元素A，根据第二个数组中的相应元素B，计算 A^B
mod	元素级的求模运算
copysign	将第二个数组中的值的符号复制给第一个数组中的值
equal,not_equal	执行元素级的比较运算，产生布尔型数组

通用函数-实例

- 将同学的考试成绩转换成整数形式的十分制分数

```
>>> np.floor(scores/10)
array([[ 7.,  8.,  7.,  9.,  8.,  8.,  8.],
       [ 6.,  6.,  8.,  7.,  8.,  9.,  9.],
       [ 9.,  9.,  8.,  8.,  8.,  9.,  9.],
       [ 8.,  8.,  9.,  8.,  8.,  8.,  8.],
       [ 8.,  7.,  7.,  9.,  9.,  7.,  8.]])
```

- 为每个同学的分数减去3分

```
>>> np.subtract(scores, 3)
array([[67, 82, 74, 87, 79, 81, 86],
       [57, 61, 77, 72, 77, 89, 87],
       [87, 90, 85, 84, 83, 87, 88],
       [77, 79, 88, 85, 80, 83, 77],
       [85, 69, 75, 87, 88, 70, 77]])
```

聚合函数

```
array([[70, 85, 77, 90, 82, 84, 89],  
       [60, 64, 80, 75, 80, 92, 90],  
       [90, 93, 88, 87, 86, 90, 91],  
       [80, 82, 91, 88, 83, 86, 80],  
       [88, 72, 78, 90, 91, 73, 80]])
```

- 支持在行、列或者全体数组元素上的聚集函数

函数	描述
sum	求和
mean	算术平均值
min、max	最大值和最小值
argmin、 argmax	最大值和最小值的索引
cumsum	从0开始向前累加各元素
cumprod	从1开始向前累乘各元素

- 统计不同科目的成绩总分

```
>>> scores.sum(axis = 0)           #按列求和  
array([388, 396, 414, 430, 422, 425,  
       430])
```
- 求“王微”所有课程成绩的平均分

```
>>> scores[names == '王微'].mean()  
82.428571428571431
```
- 查询英语考试成绩最高同学的姓名

```
>>> names[scores[:, subjects == 'English'].argmax()]  
'方绮雯'
```


随机数组生成函数

- np.random模块

函数	描述
random	随机产生[0,1)之间的浮点值
randint	随机生成给定范围内的一组整数
uniform	随机生成给定范围内服从均匀分布的一组浮点数
choice	在给定的序列内随机选择元素
normal	随机生成一组服从给定均值和方差的正态分布随机数

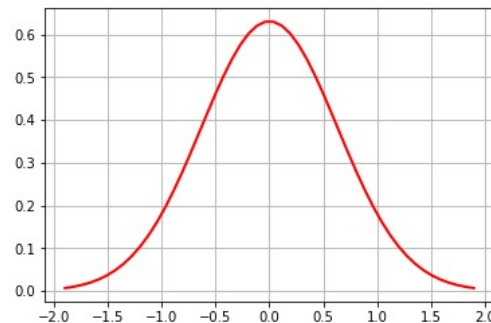
随机数组生成函数-实例

- 模块生成由10个随机整数组成的 一维数组, 整数的取值范围从0到5

- `randint(start, end, size)`

```
>>> np.random.randint(0,6,10)
array([5, 5, 0, 2, 4, 3, 1, 2, 5, 4])
```

- 生成服从均值为0、方差为1的服从**正态分布**的4×5二维数组



```
>>> np.random.normal( 0,1, size = (4,5) )
array([[ 0.50293855, -0.65924346,  1.10370417,  0.97295644, -0.94182097],
       [-0.10743896, -0.62138498, -0.70710979, -0.31265519, -0.10357636],
       [ 1.32282187,  0.91143092, -1.1728774 ,  0.51703585, -1.38091545],
       [-2.02050138, -0.936194 ,  1.47082363,  1.73261098, -0.42447148]])
```

思考与练习

1. 将scores数组中所有同学的英语成绩减去3分并显示。
2. 统计scores数组中每名同学所有科目的平均分并显示。
3. 使用随机函数生成 $[-1,1]$ 之间服从均匀分布的 3×4 二维数组，并计算所有元素的和。

案例：随机游走轨迹模拟

- 随机游走 (Random Walk) 又称随机游动或随机漫步
 - 花粉的布朗运动、证券的涨跌
- 结合NumPy的随机数生成函数与ndarray实现
 - 初始位置：(0, 0)
 - 每一步随机地沿着x轴方向左移或右移一个单位,
 - 同时沿着y轴方向左移或右移一个单位
 - 左移或者右移的概率是相等的



1. 模拟每步游走方向

- 创建一个 $2 \times n$ 的二维数组
 - 2行分别表示x、y轴上的运动
 - n 为移动总步数，每列记录一步
 - 数组元素取值为-1或1

随机整数生成在-1, 0, 1之间有3个整数，如何跳掉0？

- 假设某次随机游走走了10步
 - 用randint函数随机生成每步走的方向

生成0~1之间的整数随机数，然后将0转换为-1

```
>>> steps = 10
>>> rndwlk = np.random.randint(0, 2, size = (2, steps))
>>> rndwlk
array([[ 0,  1,  1,  1,  1,  1,  1,  0,  0,  1],
       [ 0,  1,  0,  1,  0,  0,  0,  0,  1,  0]])
```

每次生成的方向不一样

```
>>> rndwlk = np.where( rndwlk>0, 1, -1 )
>>> rndwlk
array([[ -1,  1,  1,  1,  1,  1,  1, -1, -1,  1],
       [ -1,  1, -1,  1, -1, -1, -1, -1,  1, -1]])
```

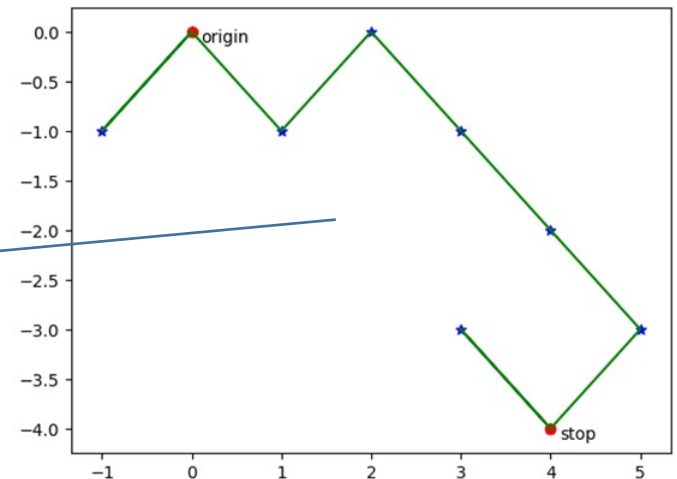
2. 计算每步游走后的位置

- 第n步的位置为前n-1步位置+第n步
 - 累加和函数cumsum()

```
>>> rndwlk
array([[ -1,  1,  1,  1,  1,  1,  1, -1, -1,  1],
       [-1,  1, -1,  1, -1, -1, -1, -1,  1, -1]])

>>> position = rndwlk.cumsum(axis = 1) #逐列求累加和
>>> position
array([[ -1,  0,  1,  2,  3,  4,  5,  4,  3,  4],
       [-1,  0, -1,  0, -1, -2, -3, -4, -3, -4]], dtype=int32)
```

实现方法，第4章，数据可视化时介绍



3. 计算每步游走后距原点距离

- 平面距离计算 $\sqrt{x^2 + y^2}$

```
>>> dists = np.sqrt(position[0]**2 + position[1]**2) #sqrt求平方根
>>> dists
array([ 1.41421356,  0. ,  1.41421356,  2. ,  3.16227766,  4.47213595,
        5.83095189,  5.65685425,  4.24264069,  5.65685425])
>>> np.set_printoptions(precision=4)
>>> dists
array([ 1.4142,  0.,  1.4142,  2. ,  3.1623,  4.4721,
        5.8310,  5.6569,  4.2426,  5.6569])
```

浮点数小数位很多，可以设置显示位数
设置后，一直有效

```
>>> dists.max() #游走最远距离
6.3246
>>> dists.min() #游走最远距离
0.0
>>> dists.mean() #游走平均距离
3.3850
```

```
>>> (dists>dists.mean()).sum() #超出平均距离
5
```

思考与练习

1. 将随机游走的步数增加到100步，计算物体距离原点的距离。
2. 重复多次随机游走过程，物体距离原点距离的变化趋势是什么？

课后作业

打包提交名为：“学号-姓名-Data-1.zip”

基于2.3节中随机游走的例子，使用ndarray和随机数生成函数模拟一个物体在三维空间随机游走的过程。

- ① 创建 3×10 的二维数组，记录物体每一步在三个轴向上的移动距离。在每个轴向上的移动距离服从标准正态分布（期望为0，方差为1）。行序0、1、2分别对应x、y和z轴；
- ② 计算每一步走完后物体在三维空间的位置；
- ③ 计算每一步走完后物体距离原点的距离；
- ④ 统计物体在z轴上到达的最远距离；（提示：使用abs()绝对值函数对z轴每一步运动后的位置求绝对值，然后求最大距离）
- ⑤ 统计物体在三维空间距离原点的最近距离值。